Projektuppgift Webbutveckling 1

Om uppgiften

Den här uppgiften är till för att du ska kunna visa vad du lärt dig i kursen. Uppgiften ska göras individuellt men det är såklart ok att fråga kompisar om råd eller hjälp. I den här uppgiften ska du anta rollen som webbutvecklare som hyrs in av en beställare för att utveckla en webbtjänst åt dem. Du ska själv bestämma vilken beställare det kan vara och vilka de tilltänkta användarna är. Du ska göra din egen design och sedan skriva kod för att bygga webbtjänsten. Bedömningskriterier för din tjänst finns längre ned i dokumentet.

Din lärare kommer vara din beställare i den här uppgiften och du kommer få visa din projektplan för läraren innan du börjar bygga tjänsten, kanske kommer du behöva göra en demonstration av din tjänst efter halva projekttiden och när det är klart så ska du redovisa projektet för din kund. Redovisningen är muntlig och innehåller en presentation av tjänsten där du visar och berättar vad den innehåller. Tjänsten ska köras på din egen dator.

Kod publiceras på GitHub så att läraren kommer åt den och skriftlig projektplan lämnas in i Classroom. Koden ska vara skriven så att kunden/läraren kan ladda ned den på sin dator och köra den, då ska alla inlänkade script, stylesheets och bilder m.m. fungera.

Projektplan

Du ska börja med att göra en projektplan. Här finns ett malldokument att utgå från. Din färdiga projektplan är 2-3 sidor och ska godkännas av kunden (läraren) innan du sätter igång att bygga din tjänst. I projektplanen beskriver du ditt projekt, vem som är kund och användare. Du gör en enkel skiss av din design i form av en eller flera wireframes. Den ska också innehålla en enkel tidplan och testplan.

Vem är kunden/beställaren?

Beskriv vem det är som betalar dig för att utveckla den här sidan. Din lärare kommer senare i projektet att anta rollen som kund/beställare.

Vem är användaren?

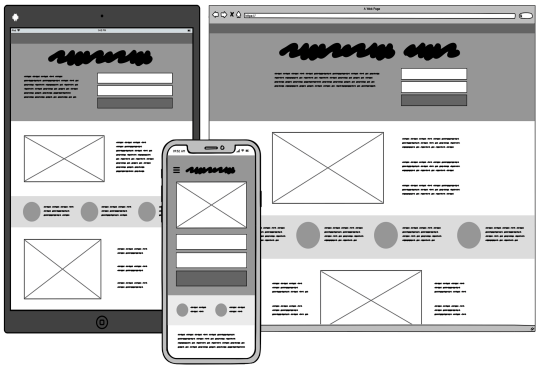
Vem är det? I vilken kontext används din sida? Är det vuxna i kontorsmiljö? Är det resande i kommunaltrafik? Är det ungdomar på fritid, vid dator, på mobil? Vad är viktigt för att din tjänst ska tilltala målgruppen?

Design

Rita en wireframe av det tänkta gränssnittet. En wireframe är en skiss av ett gränssnitt utan designelement. Den är ofta svartvit med platshållare för innehåll. Beskriv hur gränssnittet är tänkt att fungera för de tilltänkta användarna. Om *du vill* gör du wireframes för olika

skärmstorlekar. *Kravet* är att det ska finnas en wireframe för stor skärm (dator) av t.ex. din startsida.

Det finns flera gratisverktyg för att göra wireframes (t ex. https://wireframe.cc/)men välj något som du känner dig van vid att använda så att du inte lägger för mycket tid på att lära dig använda ett nytt verktyg.



Exempel på wireframes.

Planering

Det här projektet kommer följa så kallad agil utvecklingsmetodik. Vi ska se till att bygga det som är viktigast för användaren först, för att så snabbt som möjligt ha något som går att använda.

Skapa en prioriterad backlog

Du ska skapa en lista av funktioner (features) som din sajt ska ha och ordna dem med den viktigaste först. Försök skapa funktioner som är fristående, som inte beror på för många andra funktioner. Du kanske vill börja med att göra en enkel startsida för att sedan arbeta med en kontaktsida och först när den är klar lägga in animeringar på startsidan.

Exempel på prioriterade funktioner kan vara:

1. “Grundläggande design med header och footer”
2. “Meny-system”
3. “Profil-sida”
4. “Artikel-sida”
5. “Enkel kontaktsida”
6. “Karta på kontaktsida”
7. “Animation på startsida”
8. “inbäddad video på artikelsida”...

Tidplan

Hur långt uppskattar du att du kommer hinna under projektet? Hur långt har du hunnit om 6 veckor? Vad kan jag som beställare förvänta mig att se på en demo om 6 veckor?

Testplan

Beskriv hur du kommer att testa din sida. I vilka webbläsare o.s.v.. Testerna ska likna kundens tänkta användning av din sida. Om sidan i första hand är tänkt att användas på en mobil så måste testning göras för att verifiera sidans funktionalitet på mobil. Om dina tester begränsas av sidans tänkta funktionalitet (t.ex. att den bara ska användas på stora skärmar) så tar du upp det i testplanen.

Kodning av tjänst

När du arbetar med att bygga din tjänst så använd dig av din prioriterade backlog i din projektplan för att hela tiden bygga det viktigaste först. Avsluta helst en punkt i backloggen innan du går vidare.

All kod ska ligga i ett repository som ni synkar kod till **minst en gång i veckan** så att er lärare kan följa ert arbete kontinuerligt.

Vad som krävs av din tjänst framgår av bedömningskriterierna nedan.

Bedömning av kodning

Nivå G

All styling ska ske med CSS i css-fil. Alla HTML-sidorna ska använda den CSS-filen om det inte finns en bra anledning till att det inte ska vara så.

Meny: Din tjänst ska ha minst tre sidor med meny för att navigera mellan dem.

Header och footer: Alla sidor ska ha en gemensam <header> och <footer>, där emellan ska sidans huvudinnehåll finnas.

Tabell: En av sidorna ska innehålla en tabell med minst 4 rader och 2-3 kolumner. Olika rader skall ha olika färger och den översta raden skall vara rubriker för de respektive kolumn. Sista raden ska ha en styling som skiljer sig från övriga rader. Tabell-taggarna <table>, <thead>, <tbody>, <tfoot>, <th>, <tr> och <td> ska användas. All styling ska göras med CSS.

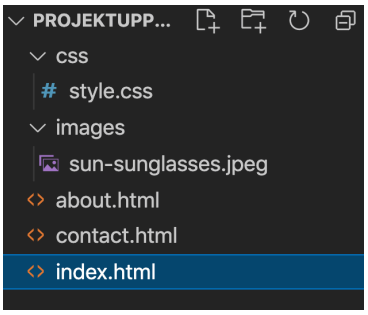
Bilder: Minst en av sidorna måste innehålla en bild (en <img>). Minst en bild ha storlek som sätts i CSS.

Rubriker ska finnas på nivå minst två nivåer (<h1> och <h2>) använd fler nivåer om det behövs.

Texten på sidorna i din tjänst behöver inte vara välformulerad. Det kan vara en låtsastext likt Lorem Ipsum eller så använder du ChatGPT för att generera din text.

Filstruktur

Alla bilder ska ligga i en mapp som heter images.
Alla css-filer ska ligga i en mapp som heter css.
Se exempel på filstruktur till höger.



Validering

HTML och CSS ska valideras. Koden ska alltså vara korrekt till syntax och struktur.
HTML ska valideras enligt HTML5.
CSS ska valideras enligt CSS3.
Se rubriken om validering längre ned i dokumentet.

Nivå VG

Allting för G-nivå ska vara uppfyllt dessutom allt beskrivet nedan.

Kontaktformulär: En sida har ett kontaktformulär med e-postadress, ärenderad och meddelandefält. Input-fälten ska vara av olika typ. Lämplig typ av inputfält ska användas. (Eller ett annat formulär där du visar hur man använder olika typer av inputfält.)

Responsivitet: Allt innehåll måste vara läsbart på en mobil enhet, ingen text får flöda “utanför skärmen”.

Extern font: Minst en extern font ska inkluderas och användas som rubriktypsnitt i minst en rubriknivå. Använd t.ex. Google Fonts

Semantiska taggar används för god tillgänglighet, minst
<nav> - för navigering
<main> - för sidans huvudinnehåll (allt mellan header och footer)
<section> - för att dela upp huvuddelen av sidan <main> i olika delar

En lista (<OL> eller <UL>) ska göras på korrekt sätt.

Alt-attribut: Alla bilder som illustrerar något (inte bakgrundsbilder) ska ha ett alt-attribut som beskriver bilden.

Bakgrundsbild: Minst ett element ska ha en bakgrundsbild.

CSS-animation: Det ska finnas minst en CSS-animation.

JavaScript: Det ska finnas minst en eventhanterare som gör något visuellt som syns för användaren.

Din kod ska vara lättläst:

- Namngivning av variabler, parametrar och funktioner ska vara beskrivande.
- Koden ska vara korrekt indenterad, gärna med 2 eller 4 spaces.
- Om koden är komplicerad och svår att förstå kan delar av den kommenteras för att underlätta för den som läser koden.
- Se denna sida om kodstruktur i HTML/CSS.

Nivå MVG, två inriktningar - välj en

Om du siktar på MVG kan du välja om du vill inrikta dig på HTML/CSS eller på JavaScript. Det räcker alltså att du gör EN av delarna nedan!

HTML/CSS på MVG-nivå

På den här nivån visar du med avancerade kunskaper inom anpassningar till olika skärmstorlekar (responsivitet), t.ex. flerkolumns-layout. Med fördel använder du flexbox. **srcset** och **lazy loading** ska användas.

Allting för nivå C ska vara uppfyllt dessutom MINST 5 saker beskrivet nedan.

1. "Hamburgar-meny" används på mobil enhet. När användaren klickar på den så menyalternativen visas och vara klickbara länkar.
2. Startsidan ska ha en "hero", antingen med video eller en bakgrundsbild.
3. Alla klickbara element är tillräckligt stora för touch. Läs mer här: https://web.dev/accessible-tap-targets/.
4. En eventhanterade i JavaScript som manipulerar DOMen. T.ex. en knapp som växlar sidan mellan dark mode/light mode.
5. Sidan med kontaktformuläret ska innehålla en karta med t.ex. Åva Gymnasium markerad. Du ska använda dig av javascript och en karttjänst. Du kan t.ex. använda Mapbox där det är gratis att skapa konto. Du ska inte använda en iframe från GoogleMaps. Styla gärna din karta på något sätt genom att använda Mapbox JavaScript-API.
6. Bilder ska ha en upplösning som är lämplig för att visa på skärm, de ska inte vara onödigt högupplösta. Filstorleken ska inte vara för stor. Bilder behöver oftast inte vara större än 200 kB.

Kodstruktur

För MVG ska du visa att du behärskar **olika sätt** att skriva CSS-selektorer. Du ska inte ensidigt styla med hjälp av id eller klass. Du ska använda **minst en** media query i din CSS. Onödig upprepning av kod får inte finnas.

Javascript på MVG-nivå

Allting på VG-nivå ska vara uppfyllt dessutom allt beskrivet nedan.
Din javascript-kod ska vara i (minst en) extern fil i en egen mapp.
Din kod måste vänta in att DOMen laddats klart innan eventlyssnare knyts till element i DOMen.

Nedan beskrivs en uppgift där ni använder ett API. Ni **måste inte** använda ett API för att nå MVG-nivå i Javascript. Det viktiga är att ni visa kunskaper att manipulera (lägga till, ta bort,

styla) saker i DOMen med hjälp av Javascript. Prata med er lärare för att diskutera hur ni skulle kunna göra det på ett annat sätt.

Din sida kan t.ex. kommunicera med ett externt API. Du kan själv välja ett API. Några exempel är: The Movie Database API, Spoonacular (recept), Star Wars API

1. Det ska finnas en sida för sökning. Den ska visa en lista av objekt. Varje objekt ska ha, namn, ev. övrig information som är relevant för användaren samt en bild om APIet innehåller det i sitt svar. I varje objekt på sidan ska länkas till en detaljsida för just det objektet. Sökfältet ska finnas på söksidan och ev. i sidans header. Ett av menyalternativen i ska vara att ta sig till söksidan.

2. Det ska finnas en sida för att visa detaljer om ett objekt i sökresultatet. Detaljerna ska hämtas genom ytterligare ett anrop till APIet som då endast hämtar information om objektet som ska visas upp. Sidan som visar detaljer ska ha en URL som går att dela så att man kommer direkt till detaljsidan som då hämtar och visar alla detaljer om objektet. Om det finns bild med i API-svaret ska den visas på detaljsidan. Detaljsidan ska ha en queryparameter som innehåller vilket objekt den visar detaljer om t.ex. `/detail.html?id=1337`

Om kommunikation med APIet misslyckas ska användaren upplysas om det med ett bra/trevligt felmeddelande. Felmeddelanden ska vara relevanta för användaren.

Inlämning

Du ska arbeta med ett **repository på GitHub** som du ska synka förändringar till med jämna mellanrum, gärna efter varje lektion.

Du ska registrera ditt GitHub-repo i ett formulär på Classroom så att din lärare kan följa arbetet.

Eventuella instruktioner som behövs till din lärare för att kunna köra sin sida (ev. preprocessorer som ska köras, API-nycklar som behövs o.s.v.) ska bifogas i en fil som heter `readme.md`.

Projektplanen lämnas in på Classroom.

Validering

För att validera webbsidan kan du förslagsvis gå in på https://html5.validator.nu/ eller http://validator.w3.org/ för att validera HTML-filer samt gå in på https://jigsaw.w3.org/css-validator/ för att validera CSS filer. På dessa sidor kan du antingen länka till en webbsida på nätet, ladda upp en fil eller skriva in koden direkt. När du validerar får du tillbaka en lista med saker som kan vara fel och som du borde åtgärda.

Tips: Använder du VSCode kan du installera plugins (t.ex. HTMLHint och StyleLint) som validerar din kod åt dig.

Stort lycka till!